

Laboratori de GIA-ABIA

Sessió 4: Cerca local (hill climbing)

Anem a resoldre el problema de *bin packing* utilitzant cerca local.

Implementant els elements del problema

Creant una classe per representar els paràmetres d'execució

Primer crearem una classe que contindrà els paràmetres que seran constants durant tota l'execució de l'algoritme. L'utilitzarem més tard de manera que no repliquem aquesta informació en cada successor. La podeu trobar al fitxer `bin_packing_problem_parameters.py`.

```
1 class ProblemParameters(object):
2     def __init__(self, h_max: int, v_h: List[int], p_max: int,
3         c_max: int):
4         self.h_max = h_max # Alcada maxima de tots
5                             # els contenidors
6         self.v_h = v_h     # Alcades de cada paquet
7         self.p_max = p_max # Maxim numero de paquets
8         self.c_max = c_max # Maxim numero de contenidors
9
10    def __repr__(self):
11        return f"Params(h_max={self.h_max}, v_h={self.v_h},
12            p_max={self.p_max}, c_max={self.c_max})"
```

Classes pels operadors

Després crearem una classe abstracta que representa un operador pel nostre problema de bin packing. Podeu trobar el codi d'aquesta secció al fitxer `bin_packing_operators.py`

```
1 class BinPackingOperator(object):
2     pass
```

Comencem amb dos operadors proposats: `moveur_paquet` i `intercanviar_paquets`.

```
1 class MoveParcel(BinPackingOperator):
2     def __init__(self, p_i: int, c_j: int, c_k: int):
3         self.p_i = p_i
4         self.c_j = c_j
5         self.c_k = c_k
6
```

```

7     def __repr__(self) -> str:
8         return f"Move parcel {self.p_i} from container {self.
            c_j} to container {self.c_k}"
9
10    class SwapParcels(BinPackingOperator):
11        def __init__(self, p_i: int, p_j: int):
12            self.p_i = p_i
13            self.p_j = p_j
14
15        def __repr__(self) -> str:
16            return f"Swap parcels {self.p_i} and {self.p_j}"

```

Representació de l'estat

Comencem a implementar la classe de representació de l'estat. En aquesta primera versió, implementem l'estat com un vector (List) amb tantes posicions com contenidors tenim, i en cada posició d'aquest vector un conjunt d'identificadors de paquet (Set).

Ens parem en la funció de còpia (copy()) de moment. Veieu algun error?

```

1    class StateRepresentation(object):
2        def __init__(self, params: ProblemParameters,
3                     v_c: List[Set[int]]):
4            self.params = params
5            self.v_c = v_c
6
7        def copy(self) -> StateRepresentation:
8            # 1) No cal copiar el ProblemParameters, perquè
9            #     son constants a tota l'execució
10           # 2) v_c es List[Set], per tant caldria fer una
11           #     còpia recursiva
12           v_c_copy = [set_i for set_i in self.v_c]
13           return StateRepresentation(self.params, v_c_copy)

```

Aquesta funció de còpia no és correcta, ja que estem copiant només els punters als conjunts i no estem recreant cadascun d'aquests conjunts. Anem a veure quines implicacions té això. Podem fer primer una prova en una terminal Python utilitzant els tipus List i Set directament:

```

1    a = [{1}, {2}, {3}]
2    b = a.copy()
3    a[0].add(4)
4    b    # Resultat: [{1, 4}, {2}, {3}]

```

Com podem comprovar, a i b, tot i no ser el mateix objecte (perquè hem fet el copy), tenen a cada posició un apuntador al mateix objecte (al mateix conjunt). Si modifiquem un conjunt, també estem modificant el mateix conjunt de l'altre objecte.

Anem a veure un altre exemple utilitzant la nostra classe de representació de l'estat:

```
1 sr = StateRepresentation(ProblemParameters(3, [1, 1, 1, 1, 1],
      5, 5), [{0}, {1}, {2}, {3}, {4}])
2 sr2 = sr.copy()
3 sr2.v_c[0].add(10)
4 print("sr -> " + str(sr.v_c))
5 # sr -> [{0}, {1}, {2}, {3}, {4}]
6 print("sr2 -> " + str(sr2.v_c))
7 # sr2 -> [{0}, {1}, {2}, {3}, {4}]
```

Ens passa el mateix: si modifiquem un dels conjunts, aquesta modificació també afecta a l'objecte copiat.

Quan generem successors a un estat, hem de tenir especial cura en que això no ens passi o acabarem generant successors amb contingut compartit o igual.

Al fitxer `bin_packing_state.py` podeu trobar la classe completa, amb una funció `copy()` correcta i amb funcions per:

- Escriure en un string una representació llegible del contingut de l'objecte estat
- Generar operadors accessibles
- Generar un successor a partir d'un operador
- Una primera versió de l'heurística (número de contenidors)

Carregueu-lo i comproveu que la còpia funciona correctament:

```
1 sr = StateRepresentation(ProblemParameters(3, [1, 1, 1, 1, 1],
      5, 5), [{0}, {1}, {2}, {3}, {4}])
2 sr2 = sr.copy()
3 sr2.v_c[0].add(10)
4 print("sr -> " + str(sr.v_c))
5 # sr -> [{0}, {1}, {2}, {3}, {4}]
6 print("sr2 -> " + str(sr2.v_c))
7 # sr2 -> [{0}, {1}, {2}, {3}, {4}]
```

Generació de l'estat inicial

Al final del fitxer trobareu un exemple de funció de generació de l'estat inicial, segons l'estratègia de generar P contenidors, amb un paquet per contenidor:

```
1 def generate_initial_state(params: ProblemParameters)
2     -> StateRepresentation:
3     assert(params.p_max <= params.c_max)
4     v_c = [{p_i} for p_i in range(params.p_max)]
5     return StateRepresentation(params, v_c)
```

Classe de problema: connectant amb AIMA

Finalment hem de crear la classe del problema, fent herència de la classe Problem de search.py per a que pugui ser utilitzada pels algorismes de cerca d'AIMA.

Com a punt de partida, podeu utilitzar la classe implementada al fitxer bin_packing_problem.py:

```
1 class BinPackingProblem(Problem):
2     def __init__(self, initial_state: StateRepresentation):
3         super().__init__(initial_state)
4
5     def actions(self, state: StateRepresentation) -> Generator[
6         BinPackingOperator, None, None]:
7         return state.generate_actions()
8
9     def result(self, state: StateRepresentation, action:
10        BinPackingOperator) -> StateRepresentation:
11        return state.apply_action(action)
12
13    def value(self, state: StateRepresentation) -> float:
14        return -state.heuristic()
15
16    def goal_test(self, state: StateRepresentation) -> bool:
17        return False
```

En cerca local, la llibreria AIMA necessita que goal_test sigui sempre fals (és l'algoritme qui decideix quan parar) i la funció d'avaluació sigui value, que ha de ser un valor a maximitzar.

Vigileu! Aquesta implementació sempre tracta de maximitzar l'heurística. Per tant, el que farem en aquesta implementació es negar el valor de l'heurística per a guiar el hill climbing en la direcció correcta.

Exercicis proposats

- Creeu un estat inicial utilitzant `generate_initial_state`, amb alçada màxima 3, amb màxim número de paquets i contenidors 5, i amb 5 paquets d'alçada 1.
- Proveu la funció de generació d'accions amb, per exemple:
`str(list(sr.generate_actions()))`
- Proveu la funció de generació d'un successor pels diferents operadors i amb diferents paràmetres, e.g.: `print(sr.apply_action(SwapParcels(0, 1)))` o `print(sr.apply_action(MoveParcel(4, 4, 0)))`. Comproveu en els outputs que els efectes són els esperats. Per exemple, podem aconseguir reduir el número de contenidors de la sol·lució?

Executant hill climbing

Anem a provar l'algoritme de hill climbing amb una alçada màxima de 3, i amb 5 paquets i 5 contenidors:

```
1 from aima.search import hill_climbing
2 from bin_packing_problem_parameters import ProblemParameters
3 from bin_packing_problem import BinPackingProblem
4 from bin_packing_state import generate_initial_state
5
6 params = ProblemParameters(3, [1, 1, 1, 1, 1], 5, 5)
7 initial_state = generate_initial_state(params)
8 n = hill_climbing(BinPackingProblem(initial_state))
9 print(n)                # Estat final
10 print(n.heuristic())    # Valor de l'estat final
```

Executeu aquest codi en un fitxer per separat o en una terminal Python. Quin resultat us dona? Sembla una sol·lució correcta?

Exercicis proposats

- Utilitzant `timeit`, comproveu el temps d'execució.
- Imagineu ara una configuració amb una alçada màxima de 30 i amb 50 paquets i 50 contenidors. Quin resultat hauria de donar? Quants contenidors haurien de sortir en una sol·lució òptima?
- Executeu aquesta configuració amb hill climbing. Quants contenidors us surten? Com n'és d'òptima la sol·lució?
- Comproveu el temps d'execució d'aquesta configuració.

- Intenteu millorar l'heurística i el temps d'execució modificant només `StateRepresentation`.